

You can compile the above code using GHC:

```
$ ghc --make template.hs
Linking template ...
```

You can then execute the built executable, which starts the server as shown below:

```
$ ./template
Setting phasers to stun... (port 3000) (ctrl-c to quit)
```

Opening a browser with URL *localhost:3000* will render the expected HTML file. You can also verify the resultant HTML output using the Curl command as shown below:

```
$ curl localhost:3000
<h1>Haskell list</h1><ul><li>http://haskell.org/</li><li>http://learnyouahaskell.com/</li><li>http://book.realworldhaskell.org/</li></ul>
```

It is good to separate the views from the actual application code. You can move the template content to a separate file as shown below:

```
-- Haskell.hs

{-# LANGUAGE OverloadedStrings #-}

module Haskell where

import Text.Blaze.Html5

render :: Html
render = do
  html $ do
    body $ do
      h1 "Haskell list"
      ul $ do
        li "http://haskell.org"
        li "http://learnyouahaskell.com/"
        li "http://book.realworldhaskell.org/"
```

The main application code is now simplified as shown below:

```
-- template-file.hs

{-# LANGUAGE OverloadedStrings #-}

import qualified Haskell
import Web.Scotty as W
import Text.Blaze.Html
import Text.Blaze.Html.Renderer.Text

blaze :: Text.Blaze.Html.Html -> ActionM ()
```

```
blaze = W.html . renderHtml
```

```
main :: IO ()
main = scotty 3000 $ do
  get "/" $ do
    blaze Haskell.render
```

You need to place both the source files (*Haskell.hs* and *template-file.hs*) in the same top level directory, and you can then compile the *template-file.hs* file that will also compile the dependency *Haskell.hs* source file as shown below:

```
$ ghc --make template-file.hs
```

You can now run the server as follows:

```
$ ./template-file
```

Executing *template-file* produces the same output as in the case of the *template.hs* example.

```
$ curl localhost:3000
<html><body><h1>Haskell list</h1><ul><li>http://haskell.org/</li><li>http://learnyouahaskell.com/</li><li>http://book.realworldhaskell.org/</li></ul></body></html>
```

You can refer to the Scotty wiki page at <https://github.com/scotty-web/scotty/wiki> for more information.

The *clay* package is a CSS preprocessor similar to *LESS* and *Sass*. You can install it using the following Cabal command:

```
$ cabal install clay
```

Let us consider a simple CSS example to generate a list of fonts to be used in the body section of an HTML page. The corresponding Clay Haskell embedded DSL looks like what's shown below:

```
-- clay-simple.hs

{-# LANGUAGE OverloadedStrings #-}

import Clay

main :: IO ()
main = putCss exampleStyleSheet

exampleStyleSheet :: Css
exampleStyleSheet = body ? fontFamily ["Baskerville",
"Georgia", "Garamond", "Times"] [serif]
```

You can compile the above code as follows:

```
$ ghc --make clay-simple.hs
[1 of 1] Compiling Main      ( clay-simple.hs, clay-simple.o )
```